JavaScript

AJAX to make a DELETE Request

List API

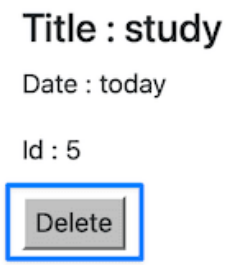
The list endpoint (API) activates the `runListGet` method.

```
app.get('/list', function(req, resp){
  runListGet(req, resp);
});

async function runListGet(req, resp) {
    try {
      const posts = db.collection(POSTS);
      const res = await posts.find().toArray();
      const query = { posts: res };
      resp.render('list.ejs', query);
    } catch (e) {
      console.error(e);
      resp.status(500).send('Error: could not read the list');
    }
}
```

list.ejs

In this code, we get all the documents in the `posts` collection, convert them into an array, and pass the array to `list.ejs`.




```
<ul class="list-group">
  <% for (var i=0; i < posts.length; i++){ %>
    <li class="list-group-item">
      <h4> Title : <%= posts[i].title %>
      </h4>
      <p> Date : <%= posts[i].date %>
      </p>
      <p> Id : <%= posts[i]._id %>
      </p>
      <button class="delete"
        data-id="<%= posts[i]._id %>">Delete</button>
    </li>
    <% } %>
</ul>
```

When we click the `delete` button, we want to delete the related document in the `posts` collection and remove the item from the page.

HTML `data-*` Attribute Conversion

An HTML `data-*` attribute is converted into the JavaScript `dataset.*` property (dashes become camelCase).

- So, the `data-id` is accessed as `dataset.id` in JavaScript.

HTML Attribute	JavaScript Property
`data-id`	`dataset.id`
`data-user-name`	`dataset.userName`
`data-post-id`	`dataset.postId`
`data-item-type`	`dataset.itemType`
`data-created-at`	`dataset.createdAt`

JavaScript/Fetch (AJAX)

In this example, we would like to use a DELETE request to the /delete endpoint (API), not to use a GET request.

- However, an HTML form cannot make a DELETE request (forms support only GET and POST).
- Instead, we use JavaScript and AJAX.
- We use JavaScript `fetch` (AJAX) to send a DELETE request with the ID of the post to be deleted.
- The delete button (with the class `delete`) invokes the AJAX call.

For the `delete` class, we attach an action that runs when the button is clicked.

```
document.addEventListener("click", async (e) => {
  if (!e.target.classList.contains("delete")) return;
```

The `e.target.dataset.id` has the ID to delete.

```
  const id = e.target.dataset.id;
```

Using JavaScript `fetch`, we call the `/delete` API of the server with the DELETE method.

```
  try {
    const res = await fetch("/delete", {
      method: "DELETE",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ _id: id })
    });

    if (!res.ok) throw new Error("Request failed");
```

/delete End Point

The JavaScript `fetch` sends the DELETE request to the `/delete` endpoint (`app.delete('/delete')`).

```
app.delete('/delete', async function(req, resp){ ...})
```

The ID arrives as a string, so we transform it into an integer.

```
const id = parseInt(req.body._id, 10);
```

Then we delete the post with that ID using the `deleteOne` method. We use `{ _id: id }` as the filter instead of `req.body`: a client could send other fields in the body and change which documents match.

```
const posts = db.collection(POSTS);
const res = await posts.deleteOne({ _id: id });
```

Fade out effect

This is JavaScript code to make the fade out effect.

```
    // Remove parent <li> with fade-out effect
    const li = e.target.closest("li");
    li.style.transition = "opacity 0.4s";
    li.style.opacity = 0;
    setTimeout(() => li.remove(), 400);
```

It finds the closest `li` element and removes it after the opacity transition.